

Report 5: Navigator Module

Part I: Complete Original System — Every Function

Part II: Every Soaring-Mode Modification and Its Impact

PX4 Autosozaring Implementation — Defence Technical Report

Fixed-Wing Autosozaring Project

April 2026

Contents

1	Role of the Navigator	2
2	Original <code>VEHICLE_CMD_DO_REPOSITION</code> Handler	2
2.1	Command Structure	2
2.2	Original Parsing Logic in <code>navigator_main.cpp</code>	2
3	Modified Navigator: Signed-Radius Turn Direction	3
3.1	Design Principle	3
3.2	New Parsing Logic	3
3.3	Mathematical Relationship: Radius and Turn Direction	3
3.4	Encoding in <code>FixedwingPositionControl</code>	3
3.5	Direction-Change Trigger	4
3.6	Latching <code>_soaring_last_clockwise</code>	4
4	<code>mission_base.cpp</code> and <code>mission.cpp</code> (Mission Support)	4
4.1	Original Mission Navigation	4
4.2	Soaring in <code>AUTO_MISSION</code> (Modes 1 and 2)	4
4.3	Transition to <code>AUTO_LOITER</code> (Modes 3 and 4)	4
5	Summary of Navigator Modifications	5

List of Symbols and Acronyms

Abbreviations

Symbol	Meaning
NED	North-East-Down coordinate frame
AMSL	Above Mean Sea Level
uORB	Micro Object Request Broker — PX4 publish/subscribe IPC
GCS	Ground Control Station
RTL	Return-To-Launch
ADS-B	Automatic Dependent Surveillance–Broadcast (aircraft transponder)
VTOL	Vertical Take-Off and Landing
NPFG	Nonlinear Path Following Guidance (used by fw_pos_control)
SP	Setpoint

Key Symbols

Symbol	Meaning	Unit
(x_c, y_c)	Loiter circle centre, NE plane	m
R	Loiter circle radius	m
ω	Loiter angular rate	rad/s
V_{TAS}	True airspeed	m/s
h	Altitude AMSL	m
h^*	Altitude setpoint carried in <code>position_setpoint_s.alt</code>	m
r_{acc}	Acceptance radius: distance at which a WP is considered reached	m
r_{alt}	Altitude acceptance radius	m
r_{loiter}	Nominal loiter radius from <code>NAV_LOITER_RAD</code>	m
v_{cruise}	Cruising speed setpoint (passed via triplet)	m/s
δ_t^*	Cruising throttle setpoint passed via triplet	—
d_{geofence}	Look-ahead test-point distance for geofence check	m

Key uORB Topics

Topic	Direction	Content
position_setpoint_triplet	Published	Previous / current / next WP setpoints
mission_result	Published	Mission validity, current WP index
navigator_status	Published	Active navigation mode, geofence status
vehicle_roi	Published	Region-of-interest target for camera/gimbal
vehicle_command_ack	Published	Acknowledgement of handled commands
vehicle_local_position	Subscribed (fd)	Local NED position and velocity
vehicle_global_position	Subscribed	Global lat/lon/alt
vehicle_status	Subscribed (fd)	Nav state, arming state, vehicle type
vehicle_command	Subscribed	Commands from Commander, GCS, pos_ctr
mission	Subscribed (fd)	Mission item list
home_position	Subscribed	Home lat/lon/alt
land_detected	Subscribed	Landed / in-air state
position_controller_status	Subscribed	Bearing, acceptance radius from FW/MC po
gps_position	Subscribed	Raw GPS for geofence check
transponder_report	Subscribed (ADS-B)	Other aircraft positions

Navigation Modes Owned by Navigator

Object	Mode
_mission (Mission)	AUTO_MISSION: follows waypoint list
_loiter (Loiter)	AUTO_LOITER: circles a fixed point
_rtl (RTL)	AUTO_RTL: return to home
_takeoff (Takeoff)	AUTO_TAKEOFF
_land (Land)	AUTO_LAND
_precland (PrecLand)	AUTO_PRECLAND
_vtol_takeoff (VtolTakeoff)	AUTO_VTOL_TAKEOFF

MAVLink Commands Handled by Navigator's run()

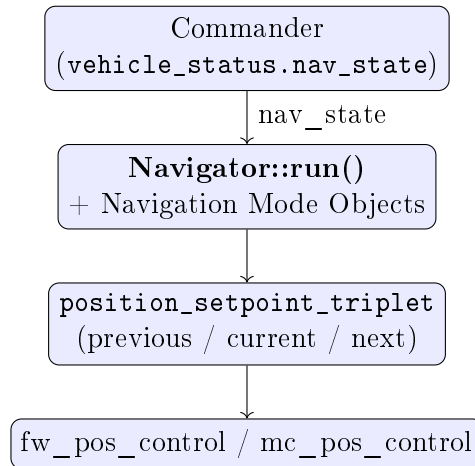
Command ID	Purpose
DO_REPOSITION (192)	Move the loiter centre, optionally switch to LOITER
DO_CHANGE_ALTITUDE (186)	Change loiter altitude
DO_ORBIT (34)	GCS-commanded circular orbit
DO_FIGUREEIGHT (35)	Figure-eight pattern (FW only)
NAV_TAKEOFF (22)	Set takeoff position
NAV_VTOL_TAKEOFF (84)	VTOL takeoff position
DO_LAND_START (189)	Jump to land sequence in mission
MISSION_START (300)	Jump to specific mission item
DO_CHANGE_SPEED (178)	Override cruising speed/throttle
DO_SET_ROI (201)	Set region of interest
DO_GO_AROUND (23)	Acknowledge go-around (handled by pos ctrl)
DO_VTOL_TRANSITION (3000)	Reset speed/throttle on VTOL transition

Part I

Original Navigator System

1 Module Purpose and Architecture

The `navigator` module is the **position setpoint manager** of PX4. It sits between the Commander (which decides the navigation *state*) and the position controllers (which track the setpoints).



Key invariant: Navigator never issues attitude or throttle commands. It only produces a `position_setpoint_triplet_s` containing:

- **previous:** last visited waypoint (for cross-track error computation).
- **current:** active waypoint/loiter centre.
- **next:** look-ahead waypoint (for smooth trajectory).

2 Navigator() Constructor

```
1 Navigator::Navigator()
```

Runs once at module start:

1. Creates the performance counter `_loop_perf`.
2. Constructs all navigation mode objects, passing `this` as the parent navigator: `_mission(this)`, `_loiter(this)`, `_rtl(this)`, `_takeoff(this)`, `_land(this)`, `_precland(this)`, `_vtol_takeoff(this)`.
3. Fills `_navigation_mode_array[]` with pointers to all modes.
4. Calls `initialize()` on every navigation mode (sets each to idle).
5. Caches parameter handles: `VT_B_DEC_MSS`, `MPC_JERK_AUTO`, `MPC_ACC_HOR` (used by RTL and path planning).

6. Opens three file-descriptor subscriptions: `vehicle_local_position`, `mission`, `vehicle_status`.
7. Advertises and initialises the `distance_sensor_mode_change_request` topic (requests distance sensor on/off as needed by landing modes).
8. Calls `reset_triplets()` to mark all setpoints invalid.

3 `params_update()`

```
1 void Navigator::params_update()
```

Called on startup and whenever `parameter_update` is received.

- `updateParams()` — reads all `DEFINE_PARAMETERS` from the module's parameter list (acceptance radii, loiter radius, RTL settings, geofence action, etc.).
- Reads parameter handles for VTOL deceleration, MPC jerk, and MPC acceleration (these are not in the navigator's own parameter list but are shared with other modules).
- Sets the mission command timeout: `_mission.set_command_timeout(_param_mis_command_timeout)`.
- On ADS-B builds: updates conflict detection parameters.

4 `run()` — The Main Control Loop

4.1 Startup (once)

1. Try to load geofence from `/fs/microsd/etc/geofence.txt`.
2. Call `params_update()`.
3. Set up three `poll()` file descriptors: `_local_pos_sub` (50 ms rate-limit), `_vehicle_status_sub`, `_mission_sub`.

4.2 Per-Cycle Steps

The loop calls `px4_poll()` with a 1000 ms timeout.

1. Copy sensor data. Every iteration:

- `orb_copy(vehicle_local_position) → _local_pos`.
- `orb_copy(vehicle_status) → _vstatus`.
- If mission fd active: copy mission, update geofence/safe-points if IDs changed.
- If GPS updated: copy `_gps_pos`.
- If global position updated: copy `_global_pos`.

2. Parameter update. If `_parameter_update_sub.updated()`: call `params_update()`.

3. Auxiliary subscriptions.

- `_land_detected_sub.update(&_land_detected)`.
- `_position_controller_status_sub.update()`.
- `_home_pos_sub.update(&_home_pos)`.

4. Vehicle command processing. Up to `ORB_QUEUE_LENGTH` commands per cycle, but only when `_wait_for_vehicle_status_timestamp == 0` (the wait mechanism prevents overwriting a reposition setpoint before the vehicle status has been refreshed after a mode switch):

1. Copy the command.
2. Dispatch to the appropriate handler (see Section ??).

5. ADS-B traffic check (if compiled in). `check_traffic()` reads transponder reports and, on conflict, calls `take_traffic_conflict_action()`.

6. Geofence breach check. `geofence_breach_check()` (Section ??).

7. Navigation mode state machine. Maps `_vstatus.nav_state` to a `NavigatorMode*` (Section ??).

8. Triplet reset on mode switch. If `_navigation_mode != navigation_mode_new`: reset the triplet, *unless* switching from takeoff to loiter with a valid loiter point (to preserve the takeoff altitude).

9. VTOL descend-to-hover override. If `nav_state == NAVIGATION_STATE_DESCEND`, vehicle is VTOL, and currently fixed-wing, and `force_vtol()` is true: send `DO_VTOL_TRANSITION` to transition back to multi-copter.

10. Run all navigation modes. Iterate `_navigation_mode_array[]`; call `mode.run(active)` for each. Only the active mode computes new setpoints; inactive modes receive `run(false)` (housekeeping only).

11. Handle null mode (no active navigation). If `_navigation_mode == nullptr`: publish an invalid triplet once.

12. Publish outputs.

- If `_pos_sp_triplet_updated`: call `publish_position_setpoint_triplet()`.
- If `_mission_result_updated`: call `publish_mission_result()`.
- Always: `publish_navigator_status()`.
- Always: `publish_distance_sensor_mode_request()`.
- Always: `_geofence.run()` (geofence logic tick).

5 Vehicle Command Handlers in run()

5.1 VEHICLE_CMD_DO_REPOSITION (192)

This is the most complex command handler. It is *only* executed when the vehicle is armed.

Wait mechanism. Before processing, a wait timestamp is set:

```
1 _wait_for_vehicle_status_timestamp = hrt_absolute_time();
```

This prevents the next command from overwriting the reposition setpoint before Commander has processed the matching mode-switch command.

Position resolution.

$$\text{lat} = \begin{cases} \text{cmd.param5} & \text{if finite} \\ \text{_global_pos.lat} & \text{otherwise (hover at current position)} \end{cases} \quad (1)$$

Same for longitude. Altitude:

$$\text{alt} = \begin{cases} \text{cmd.param7} & \text{if finite} \\ \text{_global_pos.alt} & \text{otherwise} \end{cases} \quad (2)$$

Geofence check. If the resolved position is outside the active geofence: reject with MAVLink log message; do **not** update the triplet.

Triplet construction (if inside geofence).

1. `rep->previous` ← current vehicle position (yaw, lat, lon, alt, timestamp).
2. `rep->current.type` = SETPOINT_TYPE_LOITER.
3. Ground speed: if `param1 > 0` and finite: use `param1`; else use default `get_cruising_speed()` (or `-1` on loiter entry).
4. `rep->current.cruising_throttle` = `get_cruising_throttle()`.
5. `rep->current.acceptance_radius` = `get_acceptance_radius()`.
6. Yaw: `param4` if finite, else NaN.
7. Position logic:
 - If `param5` and `param6` both finite: full position change.
 - Else if only `param7` or `param4` finite: altitude/yaw-only change (keep current loiter lat/lon).
 - Else (all NaN): pause vehicle. Fixed-wing: loiter at current position. Rotary-wing: pre-project stop point.
8. Altitude-only change branch: copy current loiter radius, minor radius, orientation, and pattern from the active setpoint.

9. Radius and direction (original code):

```
1 // ORIGINAL:
2 if (PX4_ISFINITE(cmd.param3) && cmd.param3 > 0) {
3     rep->current.loiter_radius = cmd.param3; // always positive
4     // direction: never changed from param3
5 }
```

10. Mark setpoint valid, set timestamps.

11. `_time_loitering_after_gf_breach = 0.`

5.2 VEHICLE_CMD_DO_CHANGE_ALTITUDE (186)

Identical to DO_REPOSITION with only altitude changed. Only `param1` (altitude AMSL) is used; lat/lon kept from current setpoint. Geofence check is also performed.

5.3 VEHICLE_CMD_DO_ORBIT (34) — Fixed-Wing

For fixed-wing only (multicopter handles orbits in flight tasks):

- `param1`: signed radius (negative = counter-clockwise).
- `param5/6/7`: loiter centre lat/lon/alt.
- Type: `LOITER_TYPE_ORBIT`.

Geofence check applied.

5.4 VEHICLE_CMD_DO_FIGUREEIGHT (35) — Fixed-Wing

- `param2`: minor radius of the figure-eight.
- `param1`: major radius (default: $2.5 \times$ minor radius).
- `param4`: orientation angle of the figure-eight axis.
- Pattern: `LOITER_TYPE_FIGUREEIGHT`.

Major radius enforced $\geq 2 \times$ minor radius.

5.5 VEHICLE_CMD_NAV_TAKEOFF (22)

Stores the takeoff setpoint in `_takeoff_triplet`:

- Previous: current vehicle position (if home position valid).
- Current type: `SETPOINT_TYPE_TAKEOFF`.
- Lat/lon: from `param5/6` if valid, else current position.
- Alt: `param7`.
- Yaw: NaN (handled by `FlightTaskAuto`).
- Speed: -1 (reset to default).

5.6 VEHICLE_CMD_DO_LAND_START (189)

Looks up the NAV_CMD_DO_LAND_START item in the mission. If found, sends VEHICLE_CMD_MISSION_START with the index of the next position-setpoint item after the land-start item.

5.7 VEHICLE_CMD_MISSION_START (300)

Calls `_mission.set_current_mission_index(param1)`. This causes the mission mode object to advance the active waypoint.

5.8 VEHICLE_CMD_DO_CHANGE_SPEED (178)

- If `param2 > 0`: `set_cruising_speed(param2)`.
- Else: `reset_cruising_speed()`. If `param3 > 0`: `set_cruising_throttle(param3/100)`.

Acknowledged immediately with ACCEPTED.

5.9 VEHICLE_CMD_DO_SET_ROI variants (201, 202, 195, 197, 198)

Builds and publishes `vehicle_roi_s`:

Command	ROI mode
DO_SET_ROI / NAV_ROI	Uses <code>param1</code> as mode
DO_SET_ROI_LOCATION	Fixed lat/lon/alt from <code>param5/6/7</code>
DO_SET_ROI_WPNEXT_OFFSET	Offset from next waypoint (pitch/roll/yaw)
DO_SET_ROI_NONE	Clears ROI

6 Navigation Mode State Machine

After command processing, the mode is resolved:

nav_state	Navigator object
NAVIGATION_STATE_AUTO_MISSION	<code>_mission</code>
NAVIGATION_STATE_AUTO_LOITER	<code>_loiter</code>
NAVIGATION_STATE_AUTO_RTL	<code>_rtl</code> (unless already landing)
NAVIGATION_STATE_AUTO_TAKEOFF	<code>_takeoff</code>
NAVIGATION_STATE_AUTO_VTOL_TAKEOFF	<code>_vtol_takeoff</code>
NAVIGATION_STATE_AUTO_LAND	<code>_land</code>
NAVIGATION_STATE_AUTO_PRECLAND	<code>_precland</code>
MANUAL / ACRO / ALTCTL / POSCTL / etc.	<code>nullptr</code> (no navigator output)

Disarmed guard: If `arming_state != ARMED`: force `navigation_mode_new = nullptr` regardless of `nav_state`.

RTL exception: If already in mission and landing (`_mission.isLanding()`): stay in mission mode; do not switch to RTL.

All six mode objects run every cycle, but only the active one generates setpoints. Inactive modes receive `run(false)` for internal housekeeping (e.g. resetting state machines).

7 The Position Setpoint Triplet

7.1 Structure: `position_setpoint_s`

Each setpoint in the triplet carries:

Field	Meaning
lat, lon, alt	3-D target position (WGS-84 / AMSL)
type	IDLE / POSITION / VELOCITY / LOITER / TAKEOFF
yaw	Target heading (NaN = don't care)
loiter_radius	Circle radius for loiter [m]
loiter_direction_counter_clockwise	true = CCW, false = CW
loiter_pattern	ORBIT / FIGUREEIGHT
loiter_minor_radius	Semi-minor radius for figure-eight
loiter_orientation	Figure-eight axis angle [rad]
acceptance_radius	Waypoint acceptance sphere radius [m]
alt_acceptance_radius	Altitude acceptance band [m]
cruising_speed	Override speed for this leg (-1 = default) [m/s]
cruising_throttle	Override throttle for this leg (NaN = default)
valid	Whether this setpoint should be used
timestamp	hrt time [us]

7.2 `publish_position_setpoint_triplet()`

```
1 void Navigator::publish_position_setpoint_triplet()
2 {
3     _pos_sp_triplet.timestamp = hrt_absolute_time();
4     _pos_sp_triplet_pub.publish(_pos_sp_triplet);
5     _pos_sp_triplet_updated = false;
6 }
```

Called whenever `_pos_sp_triplet_updated` is true. The consumer (`fw_pos_control`) polls this topic and rebuilds its NPGF / TECS inputs on every new triplet.

7.3 `reset_triplets()` and `reset_position_setpoint()`

```
1 void Navigator::reset_triplets()
2 {
3     reset_position_setpoint(_pos_sp_triplet.previous);
4     reset_position_setpoint(_pos_sp_triplet.current);
5     reset_position_setpoint(_pos_sp_triplet.next);
6     _pos_sp_triplet_updated = true;
7 }
```

`reset_position_setpoint(sp)` fills a setpoint with safe defaults:

- lat = lon = NaN, yaw = NaN.
- loiter_radius = `get_loiter_radius()` (from NAV_LOITER_RAD).
- acceptance_radius = `get_default_acceptance_radius()` (from NAV_ACC_RAD).

- `cruising_speed = get_cruising_speed()`.
- `cruising_throttle = get_cruising_throttle()`.
- `valid = false, type = SETPOINT_TYPE_IDLE`.
- `loiter_direction_counter_clockwise = false` (clockwise default).

8 Acceptance Radius Functions

8.1 `get_default_acceptance_radius()`

Returns `_param_nav_acc_rad.get() = NAV_ACC_RAD` (default 10 m). Used for horizontal waypoint acceptance.

8.2 `get_altitude_acceptance_radius()`

Vehicle-type dependent:

Vehicle type	Condition	Returns
Fixed-wing	<code>curr_sp.alt_acceptance_radius</code> valid	That value
Fixed-wing	Next sp is LAND and not force-VTOL	<code>NAV_FW_ALTL_RAD</code> (tighter, before landing)
Fixed-wing	Otherwise	<code>NAV_FW_ALT_RAD</code>
Rover	Any	∞ (altitude not used)
Rotary-wing	Any	<code>NAV_MC_ALT_RAD</code> (or from current sp)

8.3 `get_acceptance_radius()`

Returns `max(NAV_ACC_RAD, pos_ctrl_status.acceptance_radius)` for fixed-wing and rover (NPPFG provides its own switch-distance). For multicopter: returns `NAV_ACC_RAD`.

9 Cruising Speed and Throttle

9.1 `get_cruising_speed()`

Returns `_mission_cruising_speed` if set by a mission item (`NAV_CMD_DO_CHANGE_SPEED`), otherwise returns `-1` (use aircraft default).

9.2 `get_cruising_throttle()`

Returns `_mission_throttle` if set by a mission `DO_CHANGE_SPEED` with `param3 > 0`, otherwise NaN (use `FW_THR_TRIM`).

9.3 `set_cruising_speed() / reset_cruising_speed()`

Set/clear `_mission_cruising_speed`.

9.4 set_cruising_throttle() / reset

Set/clear `_mission_throttle`.

10 Geofence Breach Check

```
1 void Navigator::geofence_breach_check()
```

Runs at `GEOFENCE_CHECK_INTERVAL_US = 200 ms` when a geofence is active.

Test-point computation. The check is performed at a *look-ahead* point in the current direction of travel to give the aircraft time to react:

Vehicle type	Test-point distance d_{geofence}	Source
Rotary-wing	Braking distance from current velocity	<code>GFBreachAvoidance</code> model
Fixed-wing	$2r_{\text{loiter}}$	Conservative fixed radius

Geofence action (configurable via `GF_ACTION`):

<code>GF_ACTION</code>	Response
0 = NONE	Ignored
1 = WARN	MAVLink warning only
2 = LOITER	Switch to LOITER, loiter for <code>GF_MAX_HOR_DIST</code>
3 = RTL	Trigger RTL
4 = TERMINATE	Flight termination
5 = LAND	Immediate landing

If a geofence breach is detected and action $\geq$ LOITER: the navigator publishes a `geofence_result` topic and sets the failsafe trigger.

11 publish_navigator_status()

Publishes `navigator_status_s` every cycle:

- `nav_state`: current navigator mode (mirrors `vehicle_status.nav_state`).
- `failure_type`: geofence/traffic/etc. failure.
- `timestamp`.

12 abort_landing()

```
1 bool Navigator::abort_landing()
```

Returns true if:

- The current setpoint type is `SETPOINT_TYPE_LAND`; AND

- A fresh `position_controller_landing_status` has been received with `abort_status` > 0 (e.g. ground slope too steep, terrain hold timeout).

Used by the Land navigation mode to abort an approach.

13 ADS-B Traffic Handling

```
1 void Navigator::check_traffic()
```

If a new transponder report has the required validity flags (coordinates, heading, velocity, altitude):

1. `_adsb_conflict.detect_traffic_conflict()`: checks relative geometry using configurable horizontal/vertical separation thresholds.
2. If conflict detected: `take_traffic_conflict_action()` issues RTL / land / loiter depending on `NAV_TRAFF_AVOID`.
3. Stale conflicts removed after timeout.

14 Other Utility Functions

Function	Returns	Notes
<code>get_loiter_radius()</code>	<code>NAV_LOITER_RAD</code>	Default loiter radius [m]
<code>force_vtol()</code>	<code>bool</code>	True for VTOL FW in descend mode
<code>get_yaw_to_be_accepted(yaw)</code>	<code>bool</code>	True if yaw is finite
<code>load_fence_from_file()</code>	<code>void</code>	Delegates to <code>_geofence.loadFrom</code>
<code>publish_vehicle_command()</code>	<code>void</code>	Re-publishes a command for inter
<code>publish_vehicle_command_ack()</code>	<code>void</code>	Sends <code>vehicle_command_ack</code>
<code>acquire_gimbal_control()</code>	<code>void</code>	Sends gimbal-manager-take-control
<code>release_gimbal_control()</code>	<code>void</code>	Releases gimbal control
<code>stop_capturing_images()</code>	<code>void</code>	Camera-trigger stop command
<code>disable_camera_trigger()</code>	<code>void</code>	Disables camera trigger mode
<code>set_gimbal_neutral()</code>	<code>void</code>	Centres gimbal
<code>mode_completed()</code>	<code>void</code>	Called by navigation mode on co
<code>set_mission_failure_heading_timeout()</code>	<code>void</code>	Triggers mission failure timer
<code>trigger_hagl_failsafe()</code>	<code>void</code>	Height-above-ground failsafe even
<code>sendWarningDescentStoppedDueToTerrain()</code>	<code>void</code>	MAVLink warning
<code>preproject_stop_point()</code>	<code>void</code>	For multicopter, projects a brak
<code>geofence_allows_position()</code>	<code>bool</code>	True if global position is inside g

Part II

Soaring-Mode Modifications

15 Overview of Changes

Two changes were made to Navigator for soaring:

1. The DO_REPOSITION handler was extended to support a **signed radius** convention for loiter direction.
2. The NAVIGATION_STATE_ORBIT nav state was mapped to `_loiter` in the navigation mode state machine.

All other Navigator functionality is **unchanged**.

16 Modification N1: Signed Radius in DO_REPOSITION

16.1 Physical Background: Loiter Direction

For a circular loiter of radius R centred at (x_c, y_c) in the NED North-East plane, the aircraft's position at time t is:

$$\begin{pmatrix} x(t) \\ y(t) \end{pmatrix} = \begin{pmatrix} x_c \\ y_c \end{pmatrix} + R \begin{pmatrix} \cos(\omega t + \phi_0) \\ \sin(\omega t + \phi_0) \end{pmatrix} \quad (3)$$

where ϕ_0 is the initial phase angle and the angular rate is:

$$\omega = \frac{V_{TAS}}{R} \quad [\text{rad/s}] \quad (4)$$

In the NED (North-East-Down) frame:

$$\omega = \begin{cases} +\frac{V_{TAS}}{R} & \text{counter-clockwise (positive)} \\ -\frac{V_{TAS}}{R} & \text{clockwise (negative)} \end{cases} \quad (5)$$

PX4 uses the boolean field `loiter_direction_counter_clockwise` in `position_setpoint_s` to encode this. The default is `false` (clockwise).

16.2 Original Parsing

```
1 // ORIGINAL CODE in navigator_main.cpp
2 if (PX4_ISFINITE(cmd.param3) && cmd.param3 > 0) {
3     rep->current.loiter_radius = cmd.param3; // always positive
4     // loiter_direction_counter_clockwise: never changed from param3
5 }
```

Consequence:

- Negative `param3` values were silently ignored (the condition `param3 > 0` rejected them).
- There was no way to command CCW loiter via DO_REPOSITION.
- Direction was always the navigator default (clockwise).

16.3 Modified Parsing

```

1 // MODIFIED CODE
2 // Convention:
3 //   param3 > 0 --> clockwise    (loiter_direction_counter_clockwise =
4 //   param3 < 0 --> counter-clockwise (
5 //   param3 == 0 or NaN --> use NAV_LOITER_RAD default, keep direction
6 if (PX4_ISFINITE(cmd.param3) && fabsf(cmd.param3) > FLT_EPSILON) {
7     rep->current.loiter_radius = fabsf(cmd.param3);
8     rep->current.loiter_direction_counter_clockwise = (cmd.param3 < 0.f)
9     ;
10 } else if (!only_alt_change_requested) {
11     rep->current.loiter_radius = get_loiter_radius();
12     // direction stays at navigator default (false = CW)
13 }

```

Key change:

- The condition changed from `param3 > 0` to `fabsf(param3) > FLT_EPSILON` — accepts both positive and negative values.
- `loiter_radius` is now set to the absolute value `fabsf(cmd.param3)` (always positive, as required by the loiter controller).
- `loiter_direction_counter_clockwise` is set from the sign:

$$\text{CCW} = (\text{param3} < 0) \quad (6)$$

16.4 Encoding in FixedwingPositionControl

The companion-side caller builds the signed radius before publishing `DO_REPOSITION`:

```

1 // In FixedwingPositionControl::control_auto()
2 const float sign = _autosoaring_control.loiter_clockwise ? 1.f : -1.f;
3 cmd.param3 = sign * R_eff;    // signed: + = CW, - = CCW

```

The direction semantics are thus:

loiter_clockwise	cmd.param3	Navigator result
true (CW)	$+R_{\text{eff}}$	loiter_direction_CCW = false
false (CCW)	$-R_{\text{eff}}$	loiter_direction_CCW = true

16.5 Direction-Change Re-Trigger Logic

The loiter direction is *not* re-applied automatically when the companion changes direction; the navigator only processes new `DO_REPOSITION` commands. Therefore, `fw_pos_control` must re-send `DO_REPOSITION` when the direction changes even if the position is unchanged:

```

1 // In FixedwingPositionControl::control_auto()
2 const bool direction_changed =
3     (_autosoaring_control.loiter_clockwise != _soaring_last_clockwise);
4 const bool pos_changed =
5     (first_activation && airspeed_ok_for_thermal)

```

```

6      || explicit_pos_changed
7      || direction_changed; // new: re-send even if only direction
      changed
8
9  if (pos_changed) {
10     // ... build and publish DO_REPOSITION with new signed radius
11     _soaring_last_clockwise = _autosoaring_control.loiter_clockwise;
12 }

```

`_soaring_last_clockwise` is latched after each command to prevent re-sending on every cycle.

17 Modification N2: ORBIT Nav State Maps to Loiter Mode

17.1 Original Behaviour

In the original code, `NAVIGATION_STATE_ORBIT` was not present in the navigation mode switch. It fell through to the `default` case, which set `navigation_mode_new = nullptr`. The navigator produced no setpoints in ORBIT state, effectively freezing the aircraft.

17.2 Modified Behaviour

```

1 case vehicle_status_s::NAVIGATION_STATE_ORBIT:
2     // Thermal loiter: reuse the loiter navigation mode.
3     // The companion selects the centre via DO_REPOSITION;
4     // we simply map this nav-state to the existing loiter handler.
5     _pos_sp_triplet_published_invalid_once = false;
6     navigation_mode_new = &_loiter;
7     break;

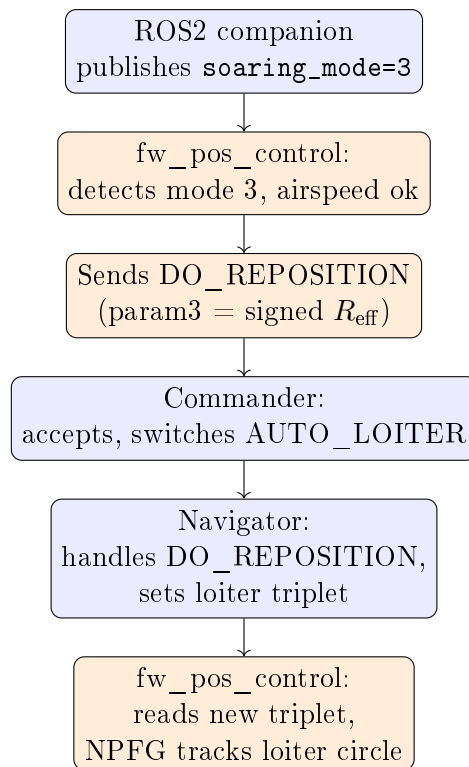
```

Why ORBIT? The standard GCS orbit command (`DO_ORBIT`) sets `nav_state = NAVIGATION_STATE_ORBIT` via Commander. By mapping ORBIT to `_loiter`, the soaring thermal mode can exploit the same orbit-request path as a standard GCS orbit command, re-using the mature loiter navigation mode without any new mode object.

17.3 Impact

- When the companion sends `soaring_mode = 3` (`THERMAL_LOITER`), `fw_pos_control` sends `DO_REPOSITION` which causes Commander to switch to `NAVIGATION_STATE_AUTO_LOITER` (the primary path). The ORBIT mapping is a safety net for the case where a GCS sends `DO_ORBIT` directly.
- No existing loiter functionality is affected; the loiter mode object `_loiter` processes either `LOITER` or `ORBIT` nav states identically.

18 Interaction Diagram: Full Thermal Loiter Flow



19 Summary Table: All Modifications to Navigator

Aspect	Original	Modified
DO_REPOSITION radius condition	<code>param3 > 0</code> only	<code>fabsf(param3) > epsilon</code> (positive or negative)
DO_REPOSITION direction	Always default (CW)	Sign of <code>param3</code> : <code>+</code> = CW, <code>-</code> = CCW
NAVIGATION_STATE_ORBITptr (no setpoints)	Not supported	Maps to <code>_loiter</code> object
Direction-change re-send	Not supported	<code>direction_changed</code> flag in <code>fw_pos_control</code> re-triggers <code>DO_REPOSITION</code>